Optimization Techniques for LLM Inference on a Multi-NPU System

Yumin Lee
School of Electrical Engineering
Korea University
Seoul, Korea
leeym@korea.ac.kr

Jeongseob Ahn
School of Electrical Engineering
Korea University
Seoul, Korea
jsahn@korea.ac.kr

ABSTRACT

The NPU is an ASIC chip specialized for matrix operations, garnering attention for exhibiting performance comparable to that of a GPU while demonstrating superior power efficiency. However, due to its fixed data flow and limited real-time scheduling capabilities, the NPU encounters various structural bottlenecks during LLM inference. Specifically, repeated requests for the same data and row buffer conflicts caused by bank contention in a multi-core system significantly degrade core utilization and operational density. These problems are difficult to resolve with a single optimization; thus, an integrated optimization approach encompassing the memory bank characteristics, data path, and scheduling is essential. This study systematically analyzes the major bottlenecks occurring in multi-NPU based LLM inference and proposes novel NPU execution optimization techniques that holistically improve data movement. The proposed techniques include: (1) Reduction of repeated data access, (2) instruction-order optimization for increased operational density, and (3) Mitigation of idle compute units. Evaluated on a Llama3-8B model and 4-core NPU environment, the proposed technique achieved a 40% speedup in the Prefill phase, during which core utilization increased significantly from 76% to 96%. In the Decoding phase, a 43% speedup was achieved even without any increased core utilization, demonstrating that the computation became more efficient. These results demonstrate that by coordinating data flow and memory access while accounting for the structural constraints of the NPU and its system, it is possible to significantly enhance LLM inference efficiency under identical hardware conditions.

1 INTRODUCTION

With the rapid growth of Large Language Model services, the demand for utilizing data center's resources for efficient inference has surged[1], [2], [3], [4]. Since LLM inference requires a significant amount of computation, it has been primarily operated on GPU systems equipped with parallel processing capabilities, and LLM service providers have been offering services through GPU servers[5], [6]. Due to the high cost and substantial power consumption of GPUs, ASIC chips, specifically Neural Processing Units (NPUs), which are designed to exclusively handle the parallel and matrix operations of GPUs, began to be researched. A key limitation of ASIC chips was that they could not change the algorithm implemented at the circuit level; thus, if the computational method of the LLM model changed, previous-generation NPUs could no longer be used. However, with the Transformer architecture maintaining the State-of-the-Art (SOTA) for a long period, the fundamental structure of LLMs has become established, making the ASIC circuit approach a realistic solution. Consequently, various attempts are being made to develop NPU ASIC chips that offer computational speeds comparable to GPUs, along with lower hardware costs and superior power efficiency

Contemporary high-performance NPU systems provide hundreds of TFLOPS of computational performance and high bandwidth via High Bandwidth Memory (HBM) to accelerate large matrix multiplications. However, due to its nature as an ASIC circuit, the NPU's fixed workflow is determined entirely at compile time, restricting its capacity to reflect real-time conditions or process dynamic requests, a capability inherent in GPUs. In Neural Processing Unit (NPU) systems, idle cores frequently arise primarily due to two factors. the Load-Execution Scheduling algorithm's inability to fully account for operational density and the inequitable distribution of computational tiles across the cores. This imbalanced distribution is rooted in the inherent challenge of designing an effective scheduler, which necessitates the simultaneous consideration of numerous interdependent variables, including diverse model sizes, sequence lengths, the number of cores, Scratchpad size, and Accumulation Scratchpad size**.** Furthermore, LLM inference exhibits a very strong memory-bound characteristic, and resource utilization is easily degraded by factors such as changes in batch size and differing computational patterns between the prefill and decoding phases. Specifically, because the NPU's Matrix Unit is specialized for GEMM operations, it possesses a structure that is not inherently amenable to the GEMV operations in the decoding phase, necessitating distinct approaches for Matrix Unit utilization between prefill and decoding.

From a memory perspective, while HBM provides high bandwidth, performance variability is significant depending on bank-level parallelism and row buffer utilization. In a multi-core

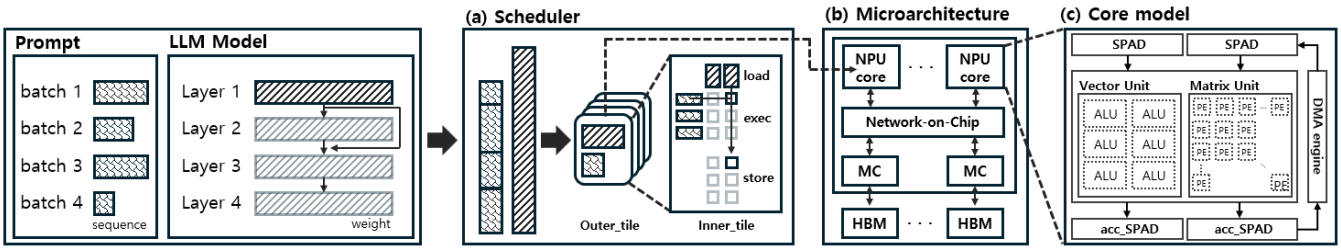


Figure 1 Overview of the Multi-NPU System with HBM. (a) Scheduler orchestrating outer/inner-tile partitioning and load–execute–store scheduling. (b) Multi-NPU microarchitecture with NPU cores connected to HBM via memory controllers and a NoC. (c) NPU core model with ScratchPAD memory(SPAD), Accumulation-ScratchPAD memory(Acc-SPAD), and vector/matrix compute units.

environment, unnecessary row conflicts frequently occur as multiple c ores attempt to access the same bank. Although HBM internally employs a policy to schedule multi-core requests to enhance row buffer utilization, the limited capacity of the request buffer is insufficient to adequately resolve the issues posed by memory-intensive workloads like LLMs. Inefficiencies also exist on the computation side. NPUs are expected to hide load time by using two Scratchpad (SPADs) to alternate between load and execution (double buffering). However, in existing structures where input/weight is loaded and flushed on a tile-by-tile basis, unnecessary SPAD flush of input data increases the volume of data loaded, leading to reduced core utilization. Moreover, improper tiling not only severely degrades core utilization by failing to ensure an even distribution of tiles per core but also lowers the operational density per data point by preventing proper latency hiding between load and execution. There is also an implementation issue within the NPU kernel. Because self-Attention operations are independent of the batch unit, GPU systems typically execute each self-attention on an independent core. However, in the NPU kernel, Self-Attention is treated as a layer, leading to scheduling as if there were inter-layer dependency, and a writeback occurs after every attention operation, resulting in insufficient utilization of the memory bandwidth.

Thus, the factors limiting LLM inference performance in NPUs arise across various layers, including: contention in multi-core systems in memory system, unnecessary SPAD flush, inefficient tiling in scheduling policy and the Systolic Array structure limitation which is inefficient for GEMV operations. Existing approaches primarily rely on cross-execution of models with different characteristics or specific architecture-specialized optimizations, making it difficult to fundamentally resolve structural bottlenecks. Especially in memory-bound LLM inference, performance improvement cannot be solely achieved by scaling the compute unit; a comprehensive optimization is required, one that jointly considers the memory hierarchy, data flow, and operational scheduling.

Therefore, this study proposes a new NPU execution efficiency technique that integrates multiple layers of optimization. memory access improvement analyzing HBM structure and scheduler characteristics to minimize row conflict, eliminating repeated data access maximizing network throughput and minimizing HBM access, simultaneous external and internal tiling optimization, stage-specific parallelization strategy for attention operations, and operational density expansion through 2-axis utilization of the systolic array during the decoding phase. By meticulously coordinating operator-level scheduling and memory access policies while considering the structural constraints of the NPU, this research aims to simultaneously enhance operational density, core utilization, and row buffer hit rate, which have been limited in existing systems this study's contributions are summarized as follows

• We identify the drawbacks of the commonly used data load patterns in LLMs and propose a technique that reduces the waste of limited memory bandwidth by eliminating unnecessary SPAD flush and by enabling the sharing of redundant activations across cores through the Inter-Core Network.

• We propose a methodology to maximize the operational density per data point through an integrated optimization approach for external and internal tiling that jointly considers SPAD, Acc-SPAD capacities, and the number of cores.

• We propose a technique to maximize the spatial locality of request data and enhance row-buffer utilization by employing HBM channel partitioning to minimize memory contention in a multi-core environment.

• Proposal of Attention Layer Parallelization Strategy: We analyze the limitations of conventional NPU kernels that process attention sequentially, unlike GPUs, and propose a method to eliminate attention layer dependency by applying distinct parallelization strategies for the prefill and decoding phases.

• To address the inefficiency of conventional structures that utilize only a single axis during the decoding phase, we propose the 2-axis systolic execution technique, which places the same input onto both axes and rearranges the weight, thereby increasing the effective computational throughput by up to 2x for the same number of FLOPs.

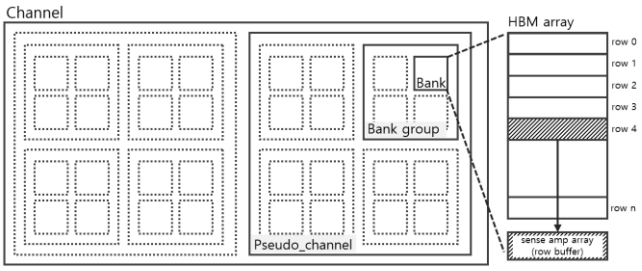


Figure 2 HBM bank hierarchy and row-buffer structure

2 BACKGROUND

2.1 Neural Processing Units

Since the LLM Model domain is saturated with Transformer models that employ the attention mechanism, various methods are being investigated to optimize large-scale matrix operations. The GPU is hardware optimized for parallel computing, which has allowed it to maximally exploit the independence of matrix multiplication in attention layer. However, the high hardware cost and substantial power consumption of GPUs impose a significant burden on inference service providers, leading to NPUs receiving considerable attention as a viable alternative to GPUs. NPUs primarily aim to deliver performance comparable to GPUs while maintaining lower hardware costs and reduced power consumption. In the calculation of large-scale matrix multiplications within LLM models, Multiply-Accumulate (MAC) units are actively employed as Processing Elements (PEs). If PEs are connected to the Memory in a point-to-point parallel fashion for parallel computation, each MAC operation for a single element of matrix multiplication would require a Load and Store operation, heavily taxing the Memory Network. The Systolic Array addresses this by arranging PEs in an N x N configuration, allowing the weight data used in one PE to be passed to an adjacent PE for reuse N times.

As shown in Figure 1(c), Google's TPU, which has achieved the state-of-the-art in NPUs, utilizes the efficient data reuse of the Systolic Array in conjunction with two Scratchpads (SPADs) employed as a double buffer [7], [9]. The TPU system manages data using a Scratchpad only instead of traditional cache hierarchy. This design choice is motivated by the fact that, unlike traditional programs, weights loaded for LLM service are often not reused, meaning performance gains from a cache hit cannot be anticipated. Furthermore, utilizing the double buffer allows each SPAD to alternate roles between execution and preloading, thereby achieving load time hiding. Additionally, operations such as normalization and activation functions are executed via a vector unit, as they require parallel execution rather than matrix operations. This structure is being utilized as the canonical architecture for NPUs.

2.2 Memory Bank

In a memory cell (such as DRAM), data storage relies on maintaining an electrical charge above a specific sense threshold on a capacitor, which is recognized as a logic '1'. However, the signal of the data stored in a Memory Cell is weak, necessitating amplification via a sense amplifier for a Read operation to be executed. Each Bank resolves this issue using a row buffer. As depicted in Figure 2, HBM is managed at the granularity of channel, pseudo channel, bank group, bank, row, and column (col). The process of loading row information into the row buffer is called row activate, and the process of initializing the data recorded in the row buffer to process new row information is termed row pre-charge. The occurrence of a request for a different row, one that is not currently in the row buffer, does not immediately trigger a pre-charge. Since each HBM channel possesses a request buffer, even if an earlier request pertains to a different row, if a request for the already open row exists in the ready queue, it is prioritized for transmission. By prioritizing requests for the currently open row buffer, the row hit rate can be increased. Generally, Cores request sequential data, allowing the maximum utilization of the spatial locality of the data prepared in the row buffer. However, this HBM structure is single-core friendly and can generate significant row conflicts in a multi-core environment [8]. To comprehend this single-core friendly memory usage, one must understand how the physical address accesses the HBM. Once the Physical address that a request will access is determined, the lower [15, 2, 2, 1, 4] bits of the physical address are used to access the memory as [row, bank, bank group, pseudo channel, col]. in a single-core environment, to prevent row conflicts, all bank and column information for one row is read first to maximize spatial locality. However, the situation is different in a multi-core environment. Even if multiple cores are executing the same program, the address locations of the data processed by the N cores are spatially distant, resulting in N rows being queued for a single bank. However, the request queue size of the HBM Channel is small, making it unable to accommodate and sort all column information of all rows to be processed by the bank. If there is no data from an activated row in the request queue, the Channel proceeds with a pre-charge-activate for the next row, leading to an excessive number of pre-charge operations.

Serving data that is present in the row buffer immediately is denoted as a row hit, which takes approximately 2 cycles. Conversely, serving data that is not present in the row buffer necessitates a sequence of pre-charge, activate, and subsequent hit operations, which collectively constitutes a row conflict and consumes 21 cycles. Due to the inherently higher frequency of row conflicts in the multi-core environment compared to single-core systems, the memory bound constraint of LLM serving is significantly intensified.

2.3 Row buffer Pre-charge Policy

When a request for a different row arrives while the current row buffer is in an activated state, the data loaded into the row buffer is pre-charged (initialized), and then the row corresponding to the new request is activated, thereby swapping the data in the row buffer. While a row hit allows for serving within 2 cycles, a row pre-charge incurs a latency of 25 cycles. Therefore, it is desirable to structure data requests to maximize the row hit rate. However, in a multi-core environment, since multiple cores simultaneously access a single bank, the number of distinct row requests that accumulate is proportional to the number of cores. HBM includes an internal request buffer that prioritizes servicing requests for the currently activated row among all requests present in the buffer. However, the buffer capacity is small, and since there is only one per HBM Channel, bank-level management is compromised. The HBM system is forced to rely solely on the data present in the request buffer to decide whether to perform a row pre-charge. If requests for the currently activated row are expected in the future, but the buffer does not currently hold them, a pre-charge occurs to change the row; subsequently, another pre-charge is needed to re-activate the original row to handle those later requests.

The HBM simulator used in the experiments, Ramulator2, was modified to address issues related to the Row Buffer Pre-charge Policy. HBM has separate schedulers at the channel, pseudo channel, bank group, and bank levels. The channel scheduler, if the request at the front of the queue is not in a ready state, swaps its position with a request in the queue that is ready and services it first. This mechanism allows a request for an activated row to be served sooner, even if it was enqueued later. This policy, however, introduces unnecessary prioritization to the row buffer pre-charge policy managed by the bank scheduler. Even if multiple row hits occur simultaneously across several banks managed by a pseudo channel, leading to a large amount of servable data, the pseudo channel data bus capacity is limited, resulting in sequential processing. This process is handled by the pseudo channel scheduler. While waiting for processing, the ready status of the activated row's request is cancelled. Since the front of the queue is not in a ready state, the queue searches for a request that is ready. If a request for an inactivated row is found, and the bank scheduler has no instruction in progress, it executes a row buffer pre-charge. Yet, the request at the queue front, which was previously being processed, accesses the same row, causing the identical row to be activated again.

To reduce unnecessary pre-charges, the policy was modified: even if the request at the front is not ready and the bank scheduler is able to execute a pre-charge instruction, the pre-charge is suppressed if a request for the currently activated row exists elsewhere in the queue. In actual HBM implementations, unnecessary pre-charging is avoided if requests for the activated row are present in the queue, making the modified scheduler a valid approach. The performance improvements applied in these experiments reflect the numerical results incorporating this revised pre-charge policy.

2.4 Model Weights Tiling

In LLM Service, the operation that consumes the most computation in both the Prefill and Decoding phases is the GEMM operation, with the usage of the Matrix Unit and Vector Unit reaching a ratio of 600 to 1. The weights used in the LLM's GEMM operation are too large to be fully loaded into the SPAD due to the substantial volume of data. Consequently, weights and activations are divided into tiles, the size of which is determined by the capacity of the SPAD. The factors considered when generating tiles are the number of cores and the sizes of the SPAD and Acc-SPAD. A standardized rule for generating tiles has not yet been established. The tiling rule in the ONNXim simulator used in these experiments was set to a 1:1 ratio for activation and weight, intended to maximize computation for the loaded data and thus efficiently utilize the ACC-SPAD [10]. However, due to a conservative policy regarding SPAD capacity, the capacities of the ACC-SPAD and SPAD were underutilized, and the number of generated tiles did not effectively reflect the number of cores available. The underutilization of the SPAD and Acc-SPAD caused by this inefficient tiling policy acts as a bottleneck in data movement, significantly impacting core utilization.

3 TECHNIQUES

In an NPU System, the speed and computational throughput of the GEMM operation are fixed. However, the overall Inference latency can be improved if Core utilization can be maximally increased by reducing the Core idle time through enhancements in data movement. We introduce methods that improve core utilization by refining the scheduling of data movement and processing, while maintaining the existing hardware specifications.

3.1 Eliminating Unnecessary SPAD Flush

3.1.1 Eliminating Unnecessary SPAD Flush

The activation and weight data required for the GEMM operation are segmented into tiles based on the SPAD size and loaded into the core. Once computation is complete, the tile is flushed from the SPAD, and the subsequent tile is loaded. Consider the K=1 and K=4 scenarios in Figure 3 in an environment where only a single NPU Core is operational. The K=1 case allows matrix computation to proceed uninterrupted within a single tile. When A(activation) and I(weight) are loaded as a single tile, the identical activation data [A] is used for the operations [AI, AM, AQ, AU]. As observed in this case, activation data [A] is flushed after the [AI] operation and subsequently re-loaded for the [AM] operation, which consumes unnecessary memory and network bandwidth. However, a simplistic approach retaining activation [A] in the SPAD and flushing it only after all weights [I, M, Q, U] becomes problematic. Specifically, in the K=4 case shown in Figure 3, flushing the activation becomes unavoidable due to the limited

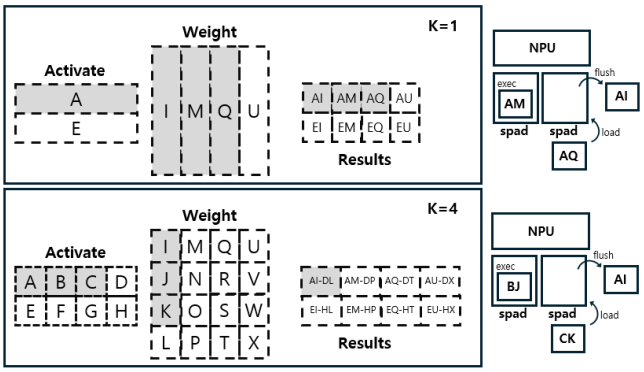


Figure 3 Activation reuse at K=1 and the impossibility of reusing the activation at K=4 due to limited SPAD resources.

SPAD resources. The core sequentially loads, flushes, and executes the tiles [AI, BJ, CK, DL] in the SPAD for accumulation matrix multiplication. Since the core has two SPADs, [AI] must be flushed to load the [CK] tile after the [AI, BJ] operations, making activation data reuse impossible due to the SPAD capacity constraint. To resolve this issue, this study proposes advancing the store timing. Conventionally, the accumulation result of the matrix multiplication, [AI, BJ, CK, DL], is stored only after all four operations are complete. Instead, we retain the loaded activations [A] and [B] in the double buffer, load/execute the subsequent weights [I/J, M/N, Q/R, U/V], and then execute an immediate writeback of the intermediate results. The intermediate calculation results written back to memory are then re-loaded when [AK, BL] are computed.

This method increases the number of writebacks but eliminates the need to re-load the activation data. Crucially, the writeback of the [AI] computation result and the data load of [AM] occur in one SPAD, while the [AJ] execution runs in the other SPAD. This configuration effectively hides the load of [AM], thus incurring no penalty for the increased writebacks. Furthermore, at the start of the first operation, only [M] (half of the [AM] data) needs to be loaded from memory, allowing the [AM] execution to commence rapidly.

3.1.2 Distributed Loading and Inter-Core Data Sharing

Now, let us consider the scenario in Figure 3 where K=1 and the number of Cores is four. Each Core is assigned [AI, AM, AQ, AU] and requests the identical Activation data [A]. The Memory must supply the same data to each Core, incurring costs in terms of Memory Traffic and Network Traffic between the Core and the Memory. The environment used in the Booksim2 simulator consists of a Node with 4 cores and 16 HBM Channels, utilizing a Flatten Butterfly Topology where Nodes, each possessing its own Router, are directly connected. As illustrated in Figure 4, by transmitting data not only through the Core-HBM connection but also via the Core-Core Network, the traffic can be effectively distributed. Furthermore, if the Cores distribute their requests for the identical Activation data to the HBM, the messages pushed to

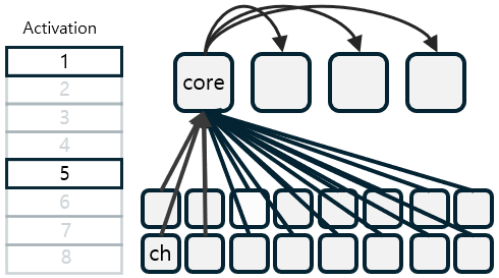


Figure 4 Distributed Loading and Inter-Core Network-Based Packet Forwarding

the HBM are issued in proportion to N/Cores. The Activation data received by one Core is then pushed to the remaining Cores, resulting in messages issued in proportion to *N*(Cores-1)/Cores*. The total number of messages pushed from the Cores remains the same before and after applying Distributed Loading. However, the volume of Requests processed by the HBM is reduced by a factor of *1/Cores*, allowing HBM to respond a faster. Additionally, Inter-Core Data Sharing does not involve processing latency, unlike sending a request to the HBM, but only transmission latency, enabling rapid sharing.

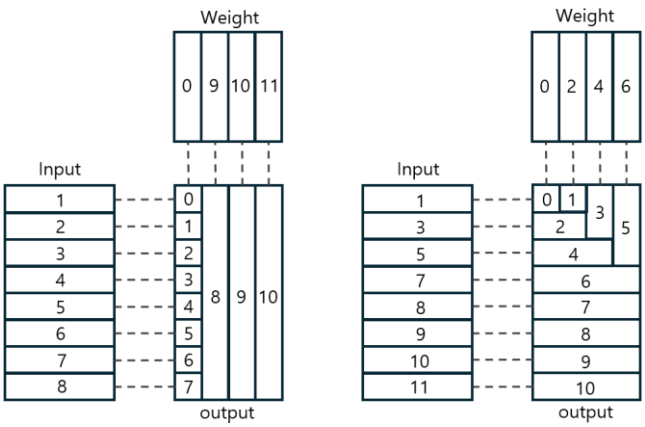
For simplified implementation, the experiment was conducted based on the assumption that the tiles are distributed evenly across the cores, which is achieved by the Outer-Tile Optimization method discussed in Section 3.2.

3.2 Tiling Optimization

When configuring tiles, two forms of optimization are considered: outer-tile optimization and inner-tile optimization. Outer-tile optimization determines how many tiles the given entire input and weight data should be divided into. Inner-tile optimization involves scheduling the data load sequence, execution, and store sequence assigned to a tile.

3.2.1 Outer-Tile Optimization

Outer-tile optimization can determine the number of tiles for the input and weight based on the size of the SPAD, Acc-SPAD, and the number of cores. The SPAD stores the data required for computation, while the Acc-SPAD stores the computation results. Therefore, optimal tile configuration is achieved by maximizing Acc-SPAD utilization while minimizing SPAD usage. Furthermore, If the number of tiles is not a multiple of the number of core, the remaining cores will remain idle during the final computation, significantly degrading core utilization. To address this inefficiency, the tiling algorithm should first maximize the tile size and then divide the overall data considering the available core count. The initial problem stems from rigidly enforcing the 1:1 activation-to-weight ratio during tile formation. This study propose an improved method: starting with the minimum computation unit size (the core height size), we iteratively and alternately increase the activation and weight dimensions by a factor of 2 until the



(a) baseline (b) inner-tile optimization

Figure 5 Inner-tile reordering to increase operational density at the start of execution

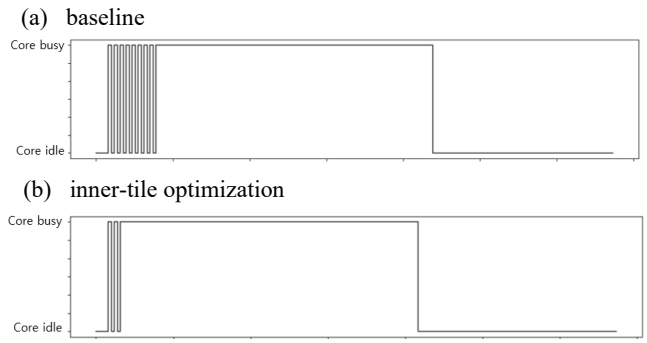


Figure 6 Increased NPU core utilization achieved by inner-tile reordering and scheduling.

resulting tile size (Activation x Weight) maximally utilizes the Acc-SPAD capacity. This condition ensures optimal Acc-SPAD utilization, maintains a proportional activation-to-weight ratio, and minimizes loaded data volume. This achieves the optimal condition for maximal execution with minimal data load, given the specified SPAD and Acc-SPAD sizes. Subsequently, the K-axis dimension is expanded to the extent permitted by the SPAD capacity. If the resulting number of tiles is not a multiple of the core count, the tiling must be re-divided such that the number of tiles in the weight dimension is closer to a multiple of the core count. However, the second problem is that simply achieving a multiple of the core count does not guarantee equitable tile distribution. As shown in Figure 3, in the K=4 case, the Matrix Unit must continuously calculate 4 tiles to perform matrix accumulation and store the result. For example, 12 tiles (K=4) can only be distributed across three cores. Therefore, when dividing tiles, only the I and J axes should be considered, excluding the K axis. This revised policy ensures maximal Acc-SPAD utilization, minimal SPAD usage, and equitable distribution across the cores

3.2.2 Inner-Tile Optimization

Inner-tile optimization is achieved by adjusting the load and execution sequence of the activation and weight data. In the conventional approach, sequentially loading input and weight often results in low operational density because the volume of computation is small relative to the loaded data. In Figure 5(a), a single weight is paired with sequentially loaded activations, and execution proceeds in the same order. Following this scheduler policy, as shown in Figure 6(a), the execution for activations (1-8) finishes before the input load completes, resulting in insufficient load hiding. However, once the second weight (9) in Figure 5(a) is loaded, it can be computed with eight activations. This allows the execution to proceed for a sufficient duration until the next weight is loaded, thereby achieving a pattern of load time hiding.

Conversely, as illustrated in Figure 5(b), if activation and weight are loaded alternately, and the computation sequence is scheduled to perform cumulative operations with the loaded data, the operational volume increases with each additional data load. This results in a higher operational density per data point. In Figure 6(b), although the initial two loads are not hidden, load hiding is executed effectively from the moment activation (3) is loaded and cumulative operation becomes possible. This demonstrates an increase in core utilization and a reduction in the total computation time.

3.3 HBM Channel Distribution per Core

The conventional memory system struggles to achieve good performance in a multi-core environment. When a core sends a request, the physical address is determined. The lower 24 bits of the physical address determine the Pseudo channel, Bank Group, Bank, Row, and Col, while specific bits of the physical address determine the accessing channel via the IPOLY hashing. Requests for consecutive physical addresses are distributed across multiple channels and banks to maximize row buffer utilization. While processing a single core's request allows for fast data retrieval by accessing multiple channels and banks in parallel, in a multi-core system, each core sends a different row request to the same bank, leading to frequent row conflicts. As shown in Figure 6(a), in a Multi-Core System, since Cores send requests to all Channels, a single Channel must simultaneously process requests from all Cores. A potential solution is to enforce that only one Core accesses each Channel. Then each bank can maximize row buffer utilization—similar to a single-core environment—thereby mitigating row conflicts. However, this approach is only effective under constrained conditions. As illustrated in Figure 7(b), in a system with 4 Cores and 16 HBM Channels, if the configuration where a single Core was connected to all 16 Channels is replaced by one where each Core is assigned an independent set of only 4 Channels, the overall memory bandwidth is reduced by a factor of

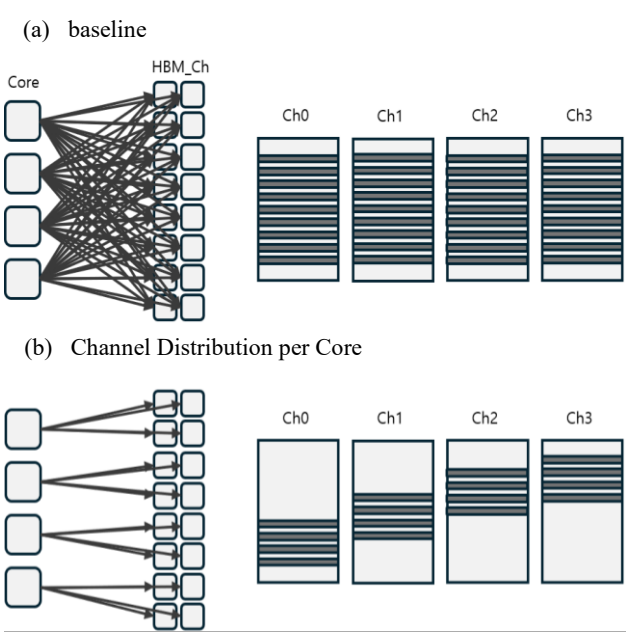


Figure 7 Channel partitioning for core-isolated access and per-channel address regions

1/4. For example, in the original environment, if only one core is loading data and the rest are idle, that core can load data rapidly from all 16 channels. Conversely, in the channel-partitioned environment, if only one core loads data, it can only load from its assigned four channels, reducing the bandwidth available to that core by 1/4. To effectively leverage channel partitioning, it is imperative that all cores load an equal amount of data, and that all cores are distributed the same number of identically sized tiles.

3.4 Parallel Attention Execution

When a multi-batch request is input on a GPU, the GEMM operation is performed on a batch-by-batch basis. The Attention operation can be computed independently for each batch and is therefore distributed across the Streaming Multiprocessors (SMs) for parallel execution. However, in NPUs, each Attention operation is conventionally treated as a single layer and processed sequentially. This leads to a writeback occurring after the completion of each attention operation, slowing down the data load for the subsequent attention operation and creating a bottleneck. During the Prefill phase, input sequences of varying lengths may be requested, potentially leading to an imbalance in the attention operations assigned to each core. Consequently, in the prefill phase, attention operations are distributed and processed across all available cores. Conversely, during the Decoding phase, a single token involves an identical volume of computation for each batch. Therefore, we propose processing one batch per core in parallel, thereby mitigating the bottleneck caused by the writeback timing.

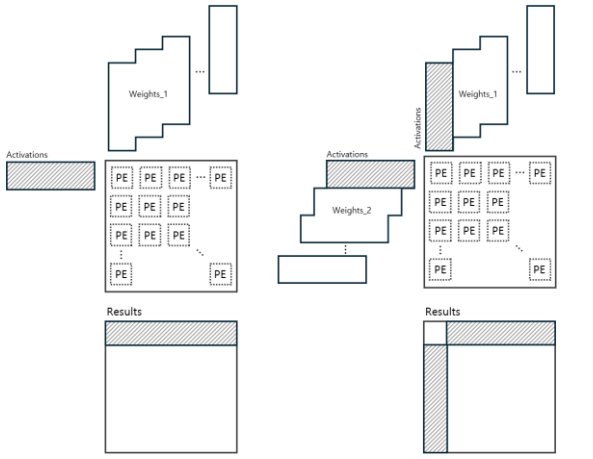


Figure 8 Increasing useful FLOPs in decoding via 2-axis systolic-array execution

3.5 2-Axis Execution

The NPU utilizes a 128x128 Matrix Unit within its Systolic Array. During the LLM Decoding phase, because a single token is provided as input, one out of the 128 Processing Elements (PEs) along the input dimension of the core produces a meaningful result as shown in Figure 8(a). The remaining PEs perform redundant computation. However, by utilizing both axes of the systolic array, as shown in Figure 8(b), placing the identical input on both axes, and transposing and placing a different weight matrix onto the orthogonal dimension, it is possible to obtain 2x more meaningful data for the same number of FLOPs, thereby reducing the computation time by half. This operation is equally applicable even when the batch size is 8.

4 IMPLEMENTATION AND EVALUATION

The experiments were conducted using the ONNXim simulator, which is capable of modeling the TPU architecture. Ramulator2 and Booksim2 were employed to simulate the HBM and the Interconnect Network, respectively [11]. [12]. The target platform for the experiments was the Llama3-8B model running on a system with the following specifications: TPU Core (Matrix Unit 128x128, Vector Unit 65536-bit, SPAD 32MB, Acc-SPAD 4MB, 1GHz) x 4, HBM2 (16 Ch, 1.2GHz), and Network (Fly Topology, 8GHz). Figure 9 compares the Prefill and Decoding Speedup and Core Utilization when the proposed prefill and decoding optimization methods were applied individually. The Speedup was measured relative to the initial Baseline (BS). The revision of the Ramulator2 Row Buffer Pre-charge Policy (PP) resulted in a 3% performance increase in Prefill and an 8% increase in Decoding. Since original simulating codes could be considered as a bug in the Ramulator2 implementation, PP was established as the new Baseline for all

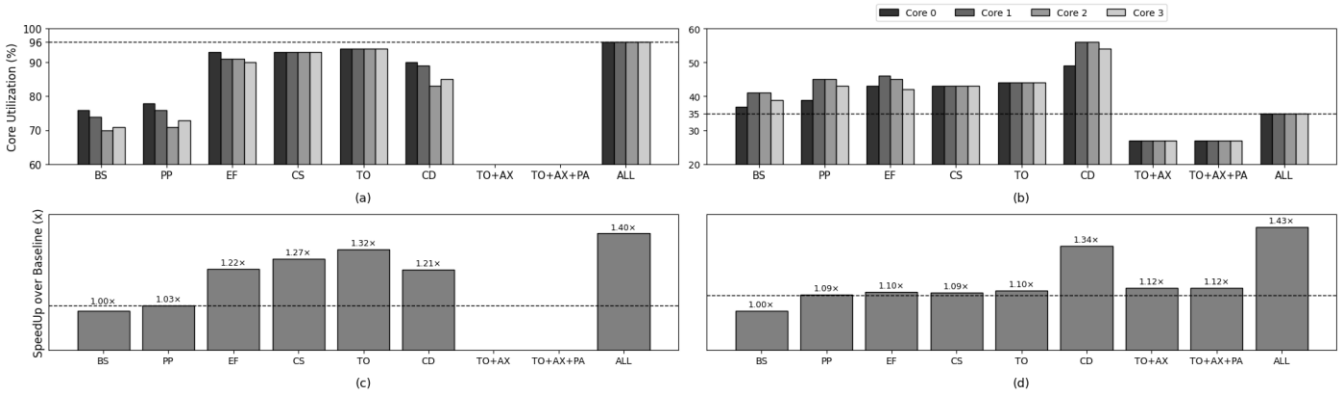


Figure 9 Comparison of NPU optimization techniques. (a) Core utilization during the prefill phase. (b) Core utilization during the decoding phase. (c) Speedup over baseline during the prefill phase. (d) Speedup over baseline during the decoding phase. BS(Baseline), PP(HBM Pre-charge Policy), EF(Eliminate Unnecessary SPAD Flush), CS(Inter-Core Network Sharing), TO(Tiling Optimization), CD(Channel Distribution), AX(2-Axis Execution), PA(Parallel Attention), ALL(All Optimizations Applied)

Table 1: Configuration of ONNXim Platforms

NPU configuration		**HBM Organization**	
		Model	HBM2
		Channel	16
Model	TPU	Bank/BankGroup	4
Systolic size/Chip	128 x 128	Bank/Channel	32
Vector Unit/Chip	65536 – bit	Capacity/Channel	1GB
SPAD size	32 MB	Frequency	1.2GHz
Acc SPAD size	4 MB	**Network Configuration**	
Frequency	1GHz	Topology	Flatten Butterfly
		Frequency	1GHz
HBM Timing Parameter (Cycle)			
nBL=2, nCL=9, nRCDRD=9, nRCDWR=9, nRP=9, nCWL=3, nWTRS=4, nWTRL=5, nRTPL=4			

subsequent experiments.

For the optimizations aimed at mitigating redundant data requests, Eliminating Unnecessary SPAD Flush (EF) and Inter-Core Network Sharing (CS) contributed to a 23% and 27% performance increase in Prefill, and a 6% and 9% increase in Decoding. The limited performance improvement of both approaches in Decoding is due to the smaller size of the activation data, which consequently reduced the amount of data available for reuse. However, in Prefill, a substantial amount of redundant Activation data could be eliminated, leading to a reduction in HBM and Network Traffic, improving Throughput, and enhancing Core Utilization.

The optimizations targeting the data loading order and path, specifically Tiling Optimization (TO) and Channel Distribution (CD), resulted in 32% and 21% performance increases in Prefill, and 8% and 33% increases in Decoding, respectively. TO proved highly effective in Prefill by successfully enabling load time hiding; however, its benefit was negligible in Decoding due to the smaller input data size, which precluded effective load hiding. CD reduced the frequency of row conflicts and maximized row buffer utilization. This memory bandwidth optimization provided substantial performance gains in both Prefill and Decoding, particularly in the memory-bound nature of the LLM service.

The 2-Axis Execution (AX) and Parallel Attention (PA) techniques, applied exclusively in Decoding, were measured cumulatively from the TO baseline to facilitate performance evaluation. TO+AX doubled the computation speed for the same data load, yielding an 11% performance improvement compared to the BS and a 1% improvement compared to the TO. Despite the 2 times increase in NPU computation speed, the necessity of loading a large volume of non-reusable weights meant that memory speed contributed more significantly to the overall speedup than the computation speed improvement, thus limiting the overall performance gain to less than the expected computational increase. TO+AX+PA achieved a 13% performance improvement compared to the BS and a 2% improvement compared to the TO baseline.

In Llama3-8B, self-attention constitutes a small proportion of the total runtime compared to major GEMM operations (QKV generation and FFN). Furthermore, the model utilizes Grouped Head Attention, which inherently reduces the computational load compared to Multi-Head Attention, consequently limiting the overall impact of Attention optimization. While the computational volume of Self-Attention increases with every generated token during Decoding, potentially exceeding FFN in long sequences, experimental time constraints led us to generate only three tokens in the Decoding Phase. This limited the full observation of the benefits of parallelizing Self-Attention. However, when performance was isolated and measured solely within the Attention Layer, PA demonstrated a 323% performance increase compared to the baseline, suggesting that a more substantial performance boost would be observed as the Decoding Phase lengthens.

5 CONCLUSIONS

This study observed that the key bottleneck determining performance in Multi-NPU based LLM Inference Services is not the computational capacity of the NPU but rather data movement. We proposed an integrated optimization methodology for data reuse, alleviating memory access, integrated tiling policy, and mitigated row buffer contension and utilization idle PE unit within a Multi-NPU System. By achieving maximal enhancement of Speedup and Core Utilization through optimization techniques, this research suggests an effective strategy for practical performance improvement in NPU-based LLM inference.

Firstly, the Eliminating Unnecessary SPAD Flush technique removes redundant flushes when Activation Reuse is possible and strengthens load–compute overlap by adjusting the timing of intermediate result writebacks. This showed significant performance improvement, especially in the Prefill segment, and this study experimentally confirmed that the double-buffer structure effectively counteracts the penalty associated with increased writebacks. Furthermore, the method enabled rapid data transfer without consuming HBM bandwidth or network traffic by utilizing Inter-Core Network Sharing for requests of identical data between Cores

Secondly, Tiling Optimization was implemented across two dimensions. Outer-Tilie Optimization was implemented with two primary objectives: to maximize SPAD and Acc-SPAD utilization by configuring the outer tiles to optimize the load-to-compute ratio and to guarantee equitable distribution by ensuring the tile count is a multiple of the core count. and Inner-Tilie Optimization aimed to increase operational density by scheduling load-execution order to achieve effective load hiding.

Thirdly, the HBM Channel Distribution per Core technique fundamentally alleviated the frequent row conflicts occurring in the multi-core environment. By assigning independent channels to each core, this approach enabled each channel to exploit row buffer locality similar to a single-core environment. Consequently, this significantly mitigated the memory-bound characteristic of LLM services, achieving high performance gains in both prefill and decoding.

Fourthly, 2-Axis Execution and Parallel Attention Execution are techniques aimed at increasing the utilization of computational resources in the Decoding Phase. In the case of 2-Axis Execution, the computation speed was doubled. However, due to the memory bound, the proportional increase in speed did not translate into a commensurate reduction in the overall execution time. The intrinsic performance improvement of Parallel Attention itself was very substantial. However, the limited computational volume of self-attention due to generating only a single token in

the simulation's Decoding phase restricted the overall latency improvement. We anticipate a more significant effect in experiments involving longer sequences or multi-token generation.

Overall, by addressing issues such as unnecessary data movement, inefficient data scheduling, and underutilization of PE units, this study achieved a speedup of 40% in the Prefill phase and 43% in the Decoding phase on the Llama3-8B model using a 4-core NPU system. Furthermore, core utilization in the Prefill phase increased from 76% to 96%, while the Decoding phase achieved performance gains without a commensurate increase in core utilization, demonstrating improved execution efficiency. These results indicate that enhancing the end-to-end dataflow, from memory, to SPAD, to Core, can substantially improve multi-NPU LLM inference performance under identical hardware conditions.

6 REFERENCES

[1] L. A. Barroso, J. Clidaras, and U. Hölzle, *The Datacenter as a Computer: An Introduction to the Design of Warehouse-Scale Machines*, Second Edition. Morgan & Claypool Publishers, 2013

[2] A. AWS, “Aws inferentia,” 2023. [Online]. Available: https://aws.amazon.com/machine-learning/inferentia/

[3] T. Chen, Z. Du, N. Sun, J. Wang, C. Wu, Y. Chen, and O. Temam, “DianNao: A Small-Footprint High-Throughput Accelerator for Ubiquitous Machine-Learning,” *in Proceedings of the 20th International Conference on Architectural Support for Programming Languages and Operating Systems* (ASPLOS’14), Salt Lake City, UT, 2014.

[4] Google, “System architecture - cloud TPU,” 2022. [Online]. Available: *https://cloud.google.com/tpu/docs/system-architecture-tpu-vm.*

[5] K. Wiggers, “Microsoft and nvidia team up to build new azure-hosted ai supercomputer,” 2022. [Online]. Available: https://techcrunch.com/2022/11/16/microsoft-and-nvidia-team-upto-build-new-azure-hosted-ai-supercomputer/

[6] M. Han, H. Zhang, R. Chen, and H. Chen, “Microsecond-scale preemption for concurrent GPU-accelerated DNN inferences,” in *Proccedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI’22)*, Carlsbad, CA, USA, 2022.

[7] Google, “System architecture - cloud TPU,” 2022. [Online]. Available: https://cloud.google.com/tpu/docs/system-architecture-tpu-vm

[8] J. Meza, J. Li, and O. Mutlu, *“Evaluating Row Buffer Locality in Future Non-Volatile Main Memories,”* SAFARI Technical Report No. 2012-002, Carnegie Mellon University, Dec. 20, 2012

[9] Thomas Norrie, Nishant Patil, Doe Hyun Yoon, George Kurian, Sheng Li, James Laudon, Cliff Young, Norman Jouppi, and David Patterson. 2021. The Design Process for Google’s Training Chips: TPUv2 and TPUv3. IEEE Micro 41, 2 (2021), 56–63. https://doi.org/10.1109/MM. 2021.3058217

[10] H. Ham, W. Yang, Y. Shin, O. Woo, G. Heo, S. Lee, J. Park, and G. Kim, *“ONNXim: A Fast, Cycle-Level Multi-Core NPU Simulator,”* IEEE Computer Architecture Letters, vol. 23, no. 2, pp. 219–222, 2024

[11] J. H. Ahn, A. Srinath, and O. Mutlu, *“Ramulator 2.0: A Fast and Extendable DRAM Simulator for Modern Workloads,”* in Proc. of the 57th IEEE/ACM International Symposium on Microarchitecture (MICRO), 2024.

[12] N. Jiang, D. U. Becker, G. Michelogiannakis, M. B. Taylor, D. E. Shaw, J. Kim, and W. J. Dally, *“A Detailed and Flexible Cycle-Accurate Network-on-Chip Simulator,”* IEEE Computer Architecture Letters, vol. 12, no. 1, pp. 21–25, 2013.